

MI JPD API

Version 2.0

© 2021 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
2.0	Initial release	12/17/2020
	Added PROCFS introduction	08/25/2021

TABLE OF CONTENTS

REVISION HISTORY	i
TABLE OF CONTENTS	ii
1. Overview	1
1.1. Module Overview	1
1.2. Decode Flow Chart	1
1.2.1 Tiramisu Decode Flow Chart	1
1.2.2 Muffin Decode Flow Chart	1
1.2.3 Mochi Decode Flow Chart	2
1.3. Keyword Description	3
1.3.1 JPG/JPEG	3
1.3.2 JPD	3
1.3.3 STR	3
1.3.4 APP	3
2. API Reference	4
2.1. MI_JPD_CreateDev	4
2.2. MI_JPD_DestroyDev	5
2.3. MI_JPD_CreateChn	6
2.4. MI_JPD_DestroyChn	8
2.5. MI_JPD_GetChnAttr	9
2.6. MI_JPD_StartChn	10
2.7. MI_JPD_StopChn	11
2.8. MI_JPD_GetChnStatus	12
2.9. MI_JPD_ResetChn	13
2.10. MI_JPD_GetStreamBuf	13
2.11. MI_JPD_PutStreamBuf	16
2.12. MI_JPD_DropStreamBuf	17
2.13. MI_JPD_QueryStreamInfo	18
2.14. MI_JPD_DirectBufDecode	19
3. Data Type	22
3.1. MI_JPD_DEV	22
3.2. MI_JPD_CHN	22
3.3. MI_JPD_ErrCode_e	23
3.4. MI_JPD_InitParam_t	23
3.5. MI_JPD_ChnCreatConf_t	24
3.6. MI_JPD_ChnAttr_t	24
3.7. MI_JPD_ChnStatus_t	25
3.8. MI_JPD_StreamBuf_t	26
3.9. MI_JPD_StreamInfo_t	28
3.10. MI_JPD_DirectInputBuf_t	29
3.11. MI_JPD_DirectOutputBuf_t	30
4. Error Code	31
5. PROCFS INTRODUCTION	32
5.1. cat	32

5.2. echo.....	34
----------------	----

1. OVERVIEW

1.1. Module Overview

JPD module provides the functions of creating decoding channel, controlling decoding, and transmitting coded streams.

There are two main types of input sources of JPD module:

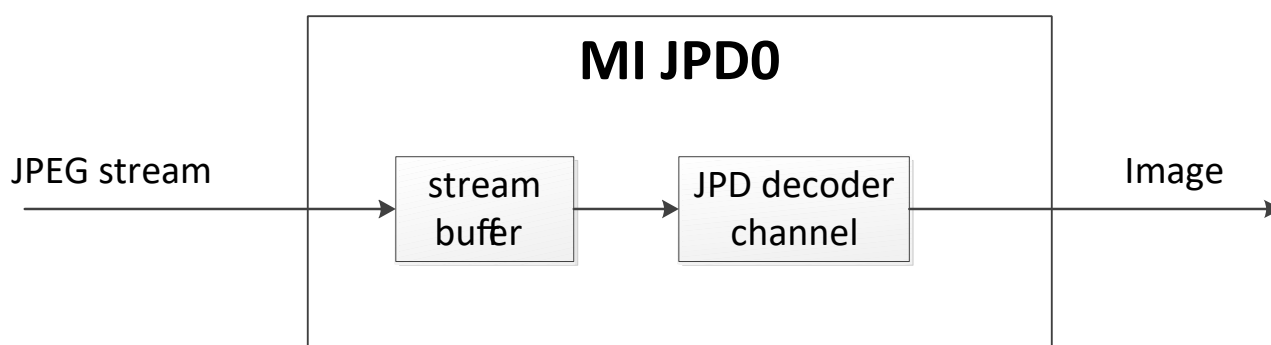
- APP reads stream file and sends data to JPD module;
- APP sends the stream data received by the network directly to JPD module.

Table 1-1 Chip decoding specifications

Chip	Max Channels	Protocol	Resolution range	Max performance
Tiramisu	16	JPEG	max 8192x8192, min 16x8	3840x2160@20fps
Muffin	16*4	JPEG	max 8192x8192, min 16x8	3840x2160@20fps
Mochi	16*2	JPEG	max 8192x8192, min 16x8	3840x2160@20fps

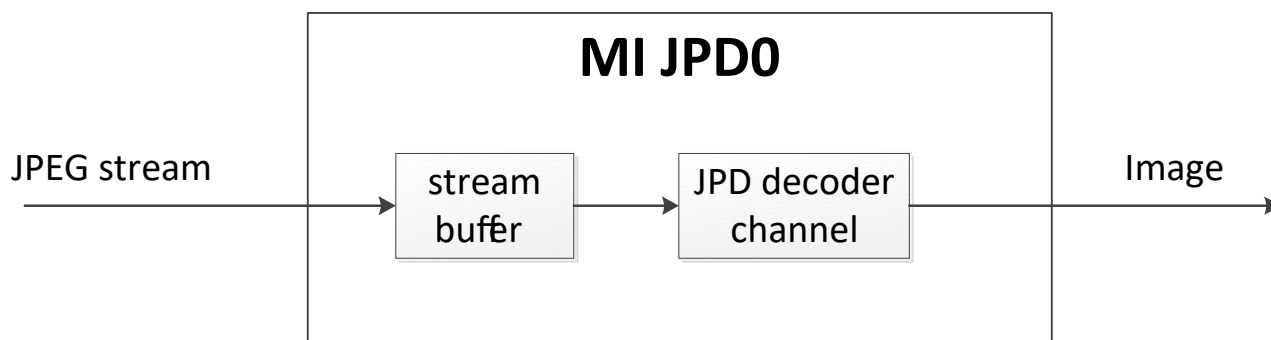
1.2. Decode Flow Chart

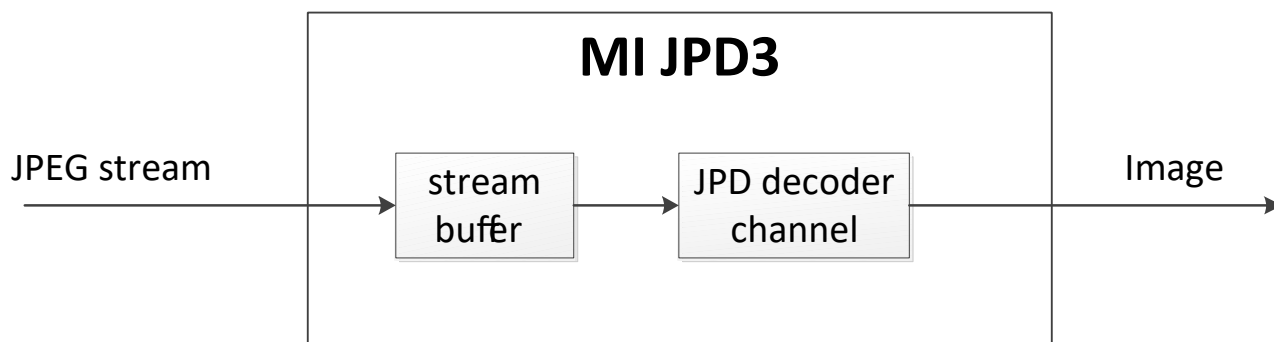
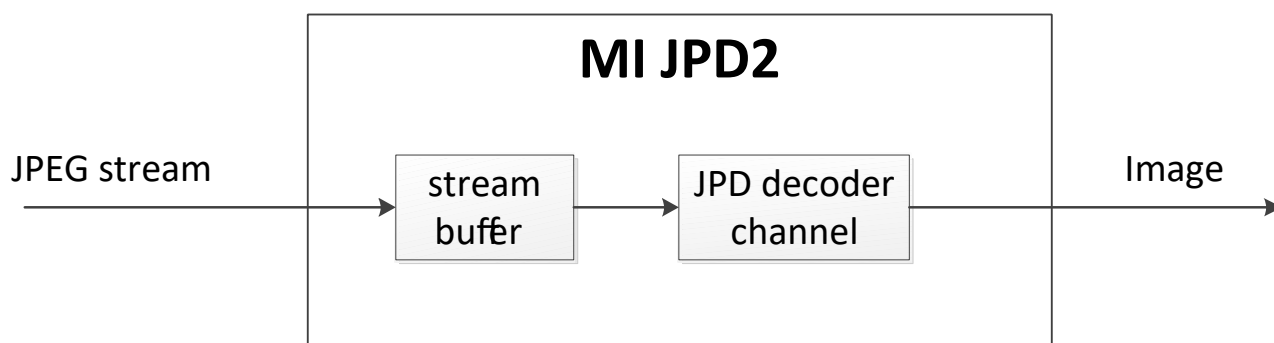
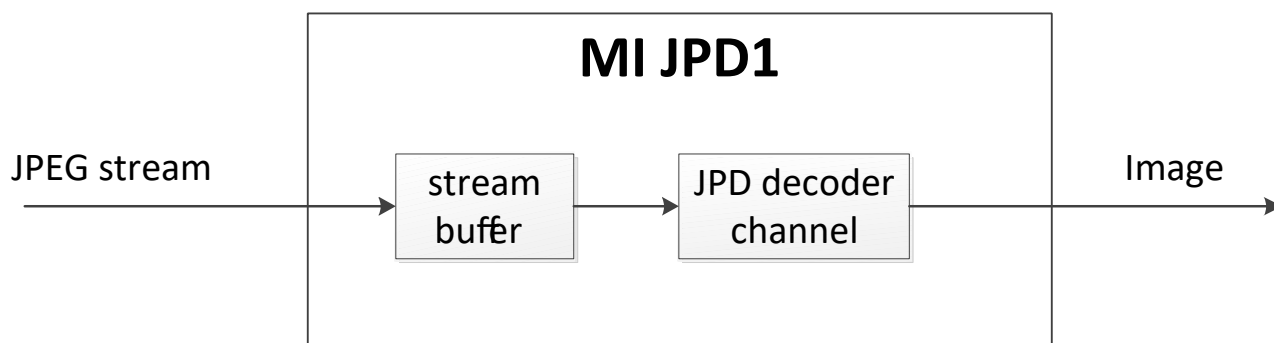
1.2.1 Tiramisu Decode Flow Chart



Support 1 device , and the id is 0.

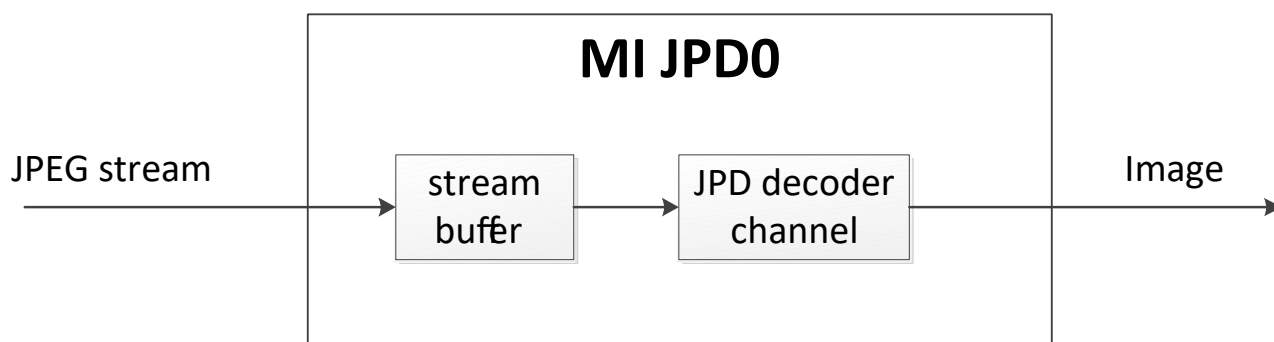
1.2.2 Muffin Decode Flow Chart

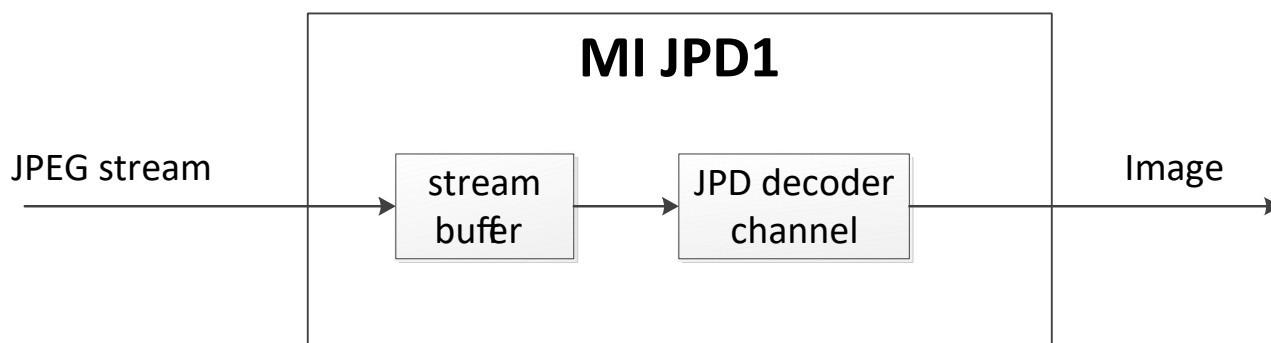




Support 4 devices, and the id are 0, 1, 2, 3.

1.2.3 Mochi Decode Flow Chart





Support 2 devices, and the id are 0, 1.

1.3. Keyword Description

1.3.1 JPG/JPEG

Joint Photographic Experts Group: the JPEG standard is developed by the organization.

1.3.2 JPD

JPG Decoder.

1.3.3 STR

Suspend To RAM: It means that after the system is shut down or enters the power saving mode, other hardware except the memory is powered off, achieving the effect of fast restart while being compatible with low power consumption.

1.3.4 APP

Application.

2. API REFERENCE

This module provides the following APIs:

Name of API	Function
MI_JPD_CreateDev	Create device
MI_JPD_DestroyDev	Destroy device
MI_JPD_CreateChn	Create a decoding channel.
MI_JPD_DestroyChn	Destroy a decoding channel.
MI_JPD_GetChnAttr	Get the decoding channel attribute.
MI_JPD_StartChn	Decoding channel starts receiving code stream.
MI_JPD_StopChn	Decoding channel stops receiving code stream.
MI_JPD_GetChnStatus	Get the decoding channel status.
MI_JPD_ResetChn	Reset the decoding channel.
MI_JPD_GetStreamBuf	Get the specified length bitstream buffer from the decoding channel.
MI_JPD_PutStreamBuf	Notify the decoding channel that the stream is put successfully and the decoding channel will start decoding.
MI_JPD_DropStreamBuf	Notify the decoding channel that the stream is dropped, and the decoding channel will release the corresponding code stream buffer obtained through MI_JPD_GetStreamBuf . MI_JPD_DropStreamBuf and MI_JPD_DropStreamBuf are mutually exclusive calls.
MI_JPD_QueryStreamInfo	Query the stream information from the decoding channel.
MI_JPD_DirectBufDecode	The decoding channel decodes directly according to the input and output buffer specified by app.

2.1. MI_JPD_CreateDev

➤ Function

Create device.

➤ Syntax

```
MI_S32 MI_JPD_CreateDev(MI_JPD_DEV JpdDev, MI_JPD_InitParam_t *pstInitParam);
```

➤ Parameter

Parameter Name	Description	Input/Output
JpdDev	Device number	Input

Parameter Name	Description	Input/Output
pstInitParam	Initialization parameters, which have no actual effect at present, are reserved for future expansion. Please refer to MI_JPD_InitParam_t .	Input

➤ Return Value

Return Value	Description
MI_SUCCESS	Successful.
MI_ERR_JPD_FAILED	Failed.
MI_ERR_JPD_INITED	The Module to be initialized is already initialized.
MI_ERR_VDEC_NULL_PTR	Null pointer.

➤ Requirement

- Header: mi_jpd.h
- Library: libmi_jpd.a/libmi_jpd.so

※ Note

- This function cannot be called, MI_JPD_CreateChn needs to specify a valid device id, and the corresponding device will be created internally.
- This interface cannot be called repeatedly, it needs to be used in a pair with [MI_JPD_DestroyDev](#), otherwise it returns to fail.
- The Muffin chip has the following restrictions when setting the MI_SYS_BindType_e parameter:
 - Only device 0 and device 1 support setting MI_SYS_BindType_e to E_MI_SYS_BIND_TYPE_REALTIME type, device 2 and device 3 do not support setting MI_SYS_BindType_e to E_MI_SYS_BIND_TYPE_REALTIME type.
 - Only 1 device is supported at the same time, MI_SYS_BindType_e can be set to E_MI_SYS_BIND_TYPE_REALTIME type, and multiple devices cannot be set to E_MI_SYS_BIND_TYPE_REALTIME type at the same time. For example: if device 0 is successfully set to E_MI_SYS_BIND_TYPE_REALTIME type, device 1 will fail to set E_MI_SYS_BIND_TYPE_REALTIME type at this time.

➤ Example

N/A.

➤ Related API

[MI_JPD_DestroyDev](#)

2.2. MI_JPD_DestroyDev

➤ Function

Destroy device.

➤ Syntax

```
MI_S32 MI_JPD_DestroyDev(MI_JPD_DEV JpdDev);
```

➤ Parameter

Parameter Name	Description	Input/Output
JpdDev	Device number	Input

➤ Return Value

Return Value	Description
MI_SUCCESS	Successful.
MI_ERR_JPD_FAILED	Failed.
MI_ERR_JPD_NOT_INIT	The Module is not initialized.

➤ Requirement

- Header: mi_jpd.h
- Library: libmi_jpd.a/libmi_jpd.so

※ Note

- This function must be called after the device is created, otherwise it returns to fail.
- If this interface is not called before the app exits, the device will be automatically destroyed internally.
- This interface cannot be called repeatedly, it needs to be used in a pair with [MI_JPD_CreateDev](#), otherwise it returns to fail.

➤ Example

N/A.

➤ Related API

[MI_JPD_CreateDev](#)

2.3. MI_JPD_CreateChn

➤ Function

Create a decoding channel.

➤ Syntax

```
MI_S32 MI_JPD_CreateChn(MI_JPD_DEV JpdDev, MI_JPD_CHN JpdChn,
MI_JPD_ChnCreatConf_t *pstChnCreatConf)
```

➤ Parameter

Parameter Name	Description	Input/Output
JpdDev	Device number	Input
JpdChn	The decoding channel number.	Input
pstChnCreatConf	The decoding channel create configuration pointer, please refer to MI_JPD_ChnCreatConf_t .	Input

➤ Return Value

Return Value	Description
MI_SUCCESS	Successful.
MI_ERR_JPD_FAILED	Failed.
MI_ERR_JPD_INVALID_CHNID	The Channel ID is out of legal range.
MI_ERR_JPD_NOT_INIT	The Module is not initialized.
MI_ERR_JPD_ILLEGAL_PARAM	Illegal parameters.
MI_ERR_JPD_CHN_EXIST	An attempt was made to create a channel that already exists.
MI_ERR_JPD_NOMEM	Failed to allocate memory (e.g. out of system memory).
MI_ERR_JPD_NULL_PTR	Null pointer.

➤ Requirement

- Header: mi_jpd.h
- Library: libmi_jpd.a/libmi_jpd.so

➤ Chip Difference

Chip	Max Decoding Channel Number
Tiramisu	16
Muffin	16

※ Note

- The channel ID cannot exceed the maximum channel number range.

➤ Example

```
MI_S32 StartJpd(void)
{
    MI_S32 s32Ret = MI_ERR_JPD_FAILED;
    MI_JPD_DEV JpdDev = 0;
    MI_JPD_CHN JpdChn = 0;
    MI_JPD_ChnCreatConf_t stChnCreatConf;

    memset(&stChnCreatConf, 0x0, sizeof(MI_JPD_ChnCreatConf_t));
    stChnCreatConf.u32StreamBufSize = 0x100000;
    stChnCreatConf.u32MaxPicWidth = 1920;
    stChnCreatConf.u32MaxPicHeight = 1080;
    s32Ret = MI_JPD_CreateChn(JpdDev, JpdChn, & stChnCreatConf);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_JPD_CreateChn failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
    }
    s32Ret = MI_JPD_StartChn(JpdDev, JpdChn);
    if(MI_SUCCESS != s32Ret)
```

```

{
    printf("MI_JPD_StartChn failed, s32Ret: 0x%x.\n", s32Ret);
    return s32Ret;
}

return s32Ret;
}

```

➤ Related API

[MI_JPD_DestroyChn](#)

2.4. MI_JPD_DestroyChn

➤ Function

Destroy a decoding channel.

➤ Syntax

MI_S32 MI_JPD_DestroyChn(MI_JPD_DEV JpdDev, MI_JPD_CHN JpdChn)

➤ Parameter

Parameter Name	Description	Input/Output
JpdDev	Device number	Input
JpdChn	The decoding channel number.	Input

➤ Return Value

Return Value	Description
MI_SUCCESS	Successful.
MI_ERR_JPD_FAILED	Failed.
MI_ERR_JPD_INVALID_CHNID	The Channel ID is out of legal range.
MI_ERR_JPD_CHN_NOT_STOP	It is not allowed to destroy the channel before stopping it.
MI_ERR_JPD_CHN_UNEXIST	The Channel does not exist.
MI_ERR_JPD_NOT_INIT	The Module is not initialized.

➤ Requirement

- Header: mi_jpd.h
- Library: libmi_jpd.a/libmi_jpd.so

※ Note

- It is not allowed to destroy the channel before stopping it.

➤ Example

```
MI_S32 StopJpd(void)
```

```
{
    MI_S32 s32Ret = MI_ERR_JPD_FAILED;
    MI_JPD_DEV JpdDev = 0;
    MI_JPD_CHN JpdChn = 0;

    // Destroy send stream thread

    s32Ret = MI_JPD_StopChn(JpdDev, JpdChn);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_JPD_StopChn failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
    }

    s32Ret = MI_JPD_DestroyChn(JpdDev, JpdChn);
    if(MI_SUCCESS != s32Ret)
    {
        printf("MI_JPD_DestroyChn failed, s32Ret: 0x%x.\n", s32Ret);
        return s32Ret;
    }

    return s32Ret;
}
```

➤ Related API

[MI_JPD_CreateChn](#)

2.5. MI_JPD_GetChnAttr

➤ function

Get the decoding channel attribute.

➤ Syntax

```
MI_S32 MI_JPD_GetChnAttr(MI_JPD_DEV JpdDev, MI_JPD_CHN JpdChn, MI_JPD_ChnAttr_t
*pstChnAttr)
```

➤ Parameter

Parameter Name	Description	Input/Output
JpdDev	Device number	Input
JpdChn	The decoding channel number.	Input
pstChnAttr	The decoding channel attribute struct pointer. Please refer to MI_JPD_ChnAttr_t .	Output

➤ Return Value

Return Value	Description
MI_SUCCESS	Successful.
MI_ERR_JPD_FAILED	Failed.
MI_ERR_JPD_INVALID_CHNID	The Channel ID is out of legal range.
MI_ERR_JPD_ILLEGAL_PARAM	Illegal parameters.
MI_ERR_JPD_CHN_UNEXIST	The Channel does not exist.
MI_ERR_JPD_NOT_INIT	The Module is not initialized.
MI_ERR_JPD_NULL_PTR	Null pointer.

- Requirement
 - Header: mi_jpd.h
 - Library: libmi_jpd.a/libmi_jpd.so

※ Note
N/A.

➤ Example
N/A.

➤ Related API
N/A.

2.6. MI_JPD_StartChn

➤ function
The channel starts receiving code stream.

➤ Syntax
MI_S32 MI_JPD_StartChn(MI_JPD_DEV JpdDev, MI_JPD_CHN JpdChn)

➤ Parameter

Parameter Name	Description	Input/Output
JpdDev	Device number	Input
JpdChn	The decoding channel number.	Input

➤ Return Value

Return Value	Description
MI_SUCCESS	Successful.
MI_ERR_JPD_FAILED	Failed.
MI_ERR_JPD_INVALID_CHNID	The Channel ID is out of legal range.
MI_ERR_JPD_CHN_UNEXIST	The Channel does not exist.
MI_ERR_JPD_NOT_INIT	The Module is not initialized.

➤ Requirement

- Header: mi_jpd.h
- Library: libmi_jpd.a/libmi_jpd.so

※ Note

- The channel must be created before starting receiving the code stream.
- Call MI_JPD_GetStreamBuf/MI_JPD_PutStreamBuf to send stream only after this API has been called first.
- This API returns success when called repeatedly.

➤ Example

Please refer to the example of [MI_JPD_CreateChn](#).

➤ Related API

[MI_JPD_StopChn](#)

2.7. MI_JPD_StopChn

➤ Function

The channel stops receiving code stream.

➤ Syntax

MI_S32 MI_JPD_StopChn(MI_JPD_DEV JpdDev, MI_JPD_CHN JpdChn)

➤ Parameter

Parameter Name	Description	Input/Output
JpdDev	Device number	Input
JpdChn	The decoding channel number.	Input

➤ Return Value

Return Value	Description
MI_SUCCESS	Successful.
MI_ERR_JPD_FAILED	Failed.
MI_ERR_JPD_INVALID_CHNID	The Channel ID is out of legal range.
MI_ERR_JPD_CHN_NOT_START	The Channel is not started.
MI_ERR_JPD_CHN_UNEXIST	The Channel does not exist.
MI_ERR_JPD_NOT_INIT	The Module is not initialized.

➤ Requirement

- Header: mi_jpd.h
- Library: libmi_jpd.a/libmi_jpd.so

※ Note

- After calling this API, the return value of MI_JPD_GetStreamBuf/MI_JPD_PutStreamBuf is MI_ERR_JPD_CHN_NOT_START.
- This API returns MI_ERR_JPD_CHN_NOT_START when called repeatedly.

➤ Example

Please refer to the example of [MI_JPD_DestroyChn](#).

➤ Related API

[MI_JPD_StartChn](#)

2.8. MI_JPD_GetChnStatus

➤ Function

Get the decoding channel status.

➤ Syntax

```
MI_S32 MI_JPD_GetChnStatus(MI_JPD_DEV JpdDev, MI_JPD_CHN JpdChn,
MI_JPD_ChnStatus_t *pstChnStatus)
```

➤ Parameter

Parameter Name	Description	Input/Output
JpdDev	Device number	Input
JpdChn	The decoding channel number.	Input
pstChnStatus	The decoding channel status struct pointer. Please refer to MI_JPD_ChnStatus_t .	Output

➤ Return Value

Return Value	Description
MI_SUCCESS	Successful.
MI_ERR_JPD_FAILED	Failed.
MI_ERR_JPD_INVALID_CHNID	The Channel ID is out of legal range.
MI_ERR_JPD_CHN_UNEXIST	The Channel does not exist.
MI_ERR_JPD_ILLEGAL_PARAM	Illegal parameters.
MI_ERR_JPD_NULL_PTR	Null pointer.

➤ Requirement

- Header: mi_jpd.h
- Library: libmi_jpd.a/libmi_jpd.so

※ Note

N/A.

➤ Example

N/A.

➤ Related API

N/A.

2.9. MI_JPD_ResetChn

➤ Function

Reset the decoding channel.

➤ Syntax

MI_S32 MI_JPD_ResetChn(MI_JPD_DEV JpdDev, MI_JPD_CHN JpdChn)

➤ Parameter

Parameter Name	Description	Input/Output
JpdDev	Device number	Input
JpdChn	The decoding channel number.	Input

➤ Return Value

Return Value	Description
MI_SUCCESS	Successful.
MI_ERR_JPD_FAILED	Failed.
MI_ERR_JPD_INVALID_CHNID	The Channel ID is out of legal range.
MI_ERR_JPD_CHN_NOT_START	The Channel is not started.
MI_ERR_JPD_CHN_UNEXIST	The Channel does not exist.

➤ Requirement

- Header: mi_jpd.h
- Library: libmi_jpd.a/libmi_jpd.so

※ Note

N/A.

➤ Example

N/A.

➤ Related API

N/A.

2.10. MI_JPD_GetStreamBuf

➤ Function

Get the specified length bitstream buffer from the decoding channel.

➤ Syntax

MI_S32 MI_JPD_GetStreamBuf(MI_JPD_DEV JpdDev, MI_JPD_CHN JpdChn, MI_U32 u32RequiredLength, MI_JPD_StreamBuf_t *pstRetStreamBuf, MI_S32 s32MilliSecToWait)

➤ Parameter

Parameter Name	Description	Input/Output
JpdDev	Device number	Input
JpdChn	The decoding channel number.	Input
u32RequiredLength	The required length of the stream buffer.	Input
pstRetStreamBuf	The stream buffer struct pointer returned by module, please refer to MI_JPD_StreamBuf_t .	Output
s32MilliSecToWait	Request timeout parameter, value range: -1: block; 0: nonblock; Positive: timeout in milliseconds.	Input

➤ Return Value

Return Value	Description
MI_SUCCESS	Successful.
MI_ERR_JPD_INVALID_CHNID	The Channel ID is out of legal range.
MI_ERR_JPD_ILLEGAL_PARAM	Illegal parameters.
MI_ERR_JPD_CHN_NOT_START	The Channel is not started.
MI_ERR_JPD_CHN_UNEXIST	The Channel does not exist.
MI_ERR_JPD_BUF_FULL	Stream buffer is full.

➤ Requirement

- Header: mi_jpd.h
- Library: libmi_jpd.a/libmi_jpd.so

※ Note

- After calling the interface successfully, copy the corresponding JPG coded stream to the address space returned by the interface. If it succeeds, call [MI_JPD_PutStreamBuf](#); if it fails, call [MI_JPD_DropStreamBuf](#).
- Only frame streaming mode is supported: a JPG bitstream of one complete frame is sent at a time.

➤ Example

```
MI_S32 JpdSendStreamThreadFunc(void)
{
    MI_S32 s32Ret = MI_ERR_JPD_FAILED;
    MI_JPD_DEV JpdDev = 0;
    MI_JPD_CHN JpdChn = 0;
    MI_U32 u32RequiredLength = 0;
    MI_S32 s32MilliSec = 0;
    MI_JPD_StreamBuf_t stRetStreamBuf;

    do {
        //Check if you need stop sending stream
        if (g_bStop)
```

```
{
    break;
}

//get one stream from file or ipc, then assign u32RequiredLength to the size of the
current stream

memset(&stRetStreamBuf, 0x0, sizeof(MI_JPD_StreamBuf_t));
s32MilliSec = 30; //timeout is 30ms
s32Ret = MI_JPD_GetStreamBuf(JpdDev, JpdChn, u32RequiredLength, &stRetStreamBuf,
s32MilliSec);
if (MI_SUCCESS != s32Ret)
{
    printf("MI_JPD_GetStreamBuf failed, s32Ret: 0x%x.\n", s32Ret);
    continue;
}

if (stRetStreamBuf.u32HeadLength+stRetStreamBuf.u32TailLength <
u32RequiredLength)
{
    //something wrong happen
    printf("MI_JPD_GetStreamBuf return wrong value: HeadLen%u TailLen%u
RequiredLength%u\n",
stRetStreamBuf.u32HeadLength, stRetStreamBuf.u32TailLength, u32RequiredLength);
    s32Ret = MI_JPD_DropStreamBuf(JpdDev, JpdChn, &stRetStreamBuf);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_JPD_DropStreamBuf failed, s32Ret: 0x%x.\n", s32Ret);
        continue;
    }
}
else
{
    memcpy(stRetStreamBuf.pu8HeadVirtAddr, addr_of_stream,
min(stRetStreamBuf.u32HeadLength, u32RequiredLength));
    if (stRetStreamBuf.u32TailLength > 0)
        memcpy(stRetStreamBuf.pu8TailVirtAddr,
addr_of_stream+stRetStreamBuf.u32HeadLength,
min(stRetStreamBuf.u32TailLength, u32RequiredLength-stRetStreamBuf.u32HeadLength));
}

stRetStreamBuf.u32ContentLength = u32RequiredLength;
s32Ret = MI_JPD_PutStreamBuf(JpdDev, JpdChn, &stRetStreamBuf);
if (MI_SUCCESS != s32Ret)
```

```

    {
        printf("MI_JPD_PutStreamBuf failed, s32Ret: 0x%x.\n", s32Ret);
        continue;
    }
}while(!g_bStop);

return MI_SUCCESS;
}

```

➤ Related API

[MI_JPD_PutStreamBuf](#)/[MI_JPD_DropStreamBuf](#)

2.11. MI_JPD_PutStreamBuf

➤ Function

Notify the decoding channel that the stream is sent successfully and the decoding channel will start decoding.

➤ Syntax

```
MI_S32 MI_JPD_PutStreamBuf(MI_JPD_DEV JpdDev, MI_JPD_CHN JpdChn,
MI_JPD_StreamBuf_t *pstRetStreamBuf)
```

➤ Parameter

Parameter Name	Description	Input/Output
JpdDev	Device number	Input
JpdChn	The decoding channel number.	Input
pstRetStreamBuf	The stream buffer pointer gotten by calling MI_JPD_GetStreamBuf , please refer to MI_JPD_StreamBuf_t .	Input

➤ Return Value

Return Value	Description
MI_SUCCESS	Successful.
MI_ERR_JPD_FAILED	Failed.
MI_ERR_JPD_INVALID_CHNID	The Channel ID is out of legal range.
MI_ERR_JPD_CHN_NOT_START	The Channel is not started.
MI_ERR_JPD_CHN_UNEXIST	The Channel does not exist.

➤ Requirement

- Header: mi_jpd.h
- Library: libmi_jpd.a/libmi_jpd.so

※ Note

- After calling MI_JPD_GetStreamBuf successfully, copy the corresponding JPG coded stream to the address space returned by MI_JPD_GetStreamBuf. If it succeeds, call

MI_JPD_Putstreambuf; if it fails, call MI_JPD_DropStreamBuf.

➤ Example

Please refer to the example of [MI_JPD_GetStreamBuf](#).

➤ Related API

[MI_JPD_GetStreamBuf](#)

2.12. MI_JPD_DropStreamBuf

➤ Function

Notify the decoding channel that the stream is dropped, and the decoding channel will release the corresponding code stream buffer obtained through [MI_JPD_GetStreamBuf](#).

➤ Syntax

```
MI_S32 MI_JPD_DropStreamBuf(MI_JPD_DEV JpdDev, MI_JPD_CHN JpdChn,
MI_JPD_StreamBuf_t *pstRetStreamBuf)
```

➤ Parameter

Parameter Name	Description	Input/Output
JpdDev	Device number	Input
JpdChn	The decoding channel number.	Input
pstRetStreamBuf	The stream buffer pointer gotten by calling MI_JPD_GetStreamBuf, please refer to MI_JPD_StreamBuf_t .	Input

➤ Return Value

Return Value	Description
MI_SUCCESS	Successful.
MI_ERR_JPD_FAILED	Failed.
MI_ERR_JPD_INVALID_CHNID	The Channel ID is out of legal range.
MI_ERR_JPD_CHN_NOT_START	The Channel is not started.
MI_ERR_JPD_CHN_UNEXIST	The Channel does not exist.

➤ Requirement

- Header: mi_jpd.h
- Library: libmi_jpd.a/libmi_jpd.so

※ Note

- After calling MI_JPD_GetStreamBuf successfully, copy the corresponding JPG coded stream to the address space returned by MI_JPD_GetStreamBuf. If it succeeds, call MI_JPD_Putstreambuf; if it fails, call MI_JPD_DropStreamBuf.

➤ Example

Please refer to the example of [MI_JPD_GetStreamBuf](#).

➤ Related API

[MI_JPD_GetStreamBuf](#)

2.13. MI_JPD_QueryStreamInfo

➤ Function

Query the stream information from the decoding channel.

➤ Syntax

```
MI_S32 MI_JPD_QueryStreamInfo(MI_JPD_DEV JpdDev, MI_JPD_CHN JpdChn,
MI_JPD_DirectInputBuf_t *pstInputBuf, MI_JPD_StreamInfo_t *pstStreamInfo)
```

➤ Parameter

Parameter Name	Description	Input/Output
JpdDev	Device number	Input
JpdChn	The decoding channel number.	Input
pstInputBuf	The struct pointer of input buffer, please refer to MI_JPD_DirectInputBuf_t .	Input
pstStreamInfo	The struct pointer of stream information, please refer to MI_JPD_StreamInfo_t .	Output

➤ Return Value

Return Value	Description
MI_SUCCESS	Successful.
MI_ERR_JPD_FAILED	Failed.
MI_ERR_JPD_INVALID_CHNID	The Channel ID is out of legal range.
MI_ERR_JPD_CHN_NOT_START	The Channel is not started.
MI_ERR_JPD_CHN_UNEXIST	The Channel does not exist.

➤ Requirement

- Header: mi_jpd.h
- Library: libmi_jpd.a/libmi_jpd.so

※ Note

The u32BufWidth/u32BufHeight refers to the actual width/height of the output buffer, while u32CurPicWidth/u32CurPicHeight refers to the effective width/height of the image.

➤ Example

Please see the example of [MI_JPD_DirectBufDecode](#).

➤ Related API

[MI_JPD_DirectBufDecode](#)

2.14. MI_JPD_DirectBufDecode

➤ Function

The decoding channel decodes directly according to the input and output buffer specified by APP.

➤ Syntax

```
MI_S32 MI_JPD_DirectBufDecode(MI_JPD_DEV JpdDev, MI_JPD_CHN JpdChn,
MI_JPD_DirectInputBuf_t *pstInputBuf, MI_JPD_DirectOutputBuf_t *pstOutputBuf)
```

➤ Parameter

Parameter Name	Description	Input/Output
JpdDev	Device number	Input
JpdChn	The decoding channel number.	Input
pstInputBuf	The struct pointer of input buffer, please refer to MI_JPD_DirectInputBuf_t .	Input
pstOutputBuf	The struct pointer of output buffer, please refer to MI_JPD_DirectOutputBuf_t .	Input

➤ Return Value

Return Value	Description
MI_SUCCESS	Successful.
MI_ERR_JPD_FAILED	Failed.
MI_ERR_JPD_INVALID_CHNID	The Channel ID is out of legal range.
MI_ERR_JPD_CHN_NOT_START	The Channel is not started.
MI_ERR_JPD_CHN_UNEXIST	The Channel does not exist.

➤ Requirement

- Header: mi_jpd.h
- Library: libmi_jpd.a/libmi_jpd.so

※ Note

- If the output buffer size is not clear, APP needs to call [MI_JPD_QueryStreamInfo](#) before calling this interface. The output buffer size is calculated by u32BufWidth/u32BufHeight of [MI_JPD_StreamInfo_t](#), while u32CurPicWidth/u32CurPicHeight refers to the actual effective width/height of the image.

➤ Example

```
MI_S32 directBufDecode(void)
{
    MI_S32 s32Ret = MI_SUCCESS;
    MI_JPD_DEV JpdDev = 0;
    MI_JPD_CHN JpdChn = 0;
    MI_U8 *pu8Buf = NULL;
```

```
MI_U32 u32LengthCurStream = 0;
MI_JPD_DirectInputBuf_t stInputBuf;
MI_JPD_DirectOutputBuf_t stOutputBuf;
MI_JPD_StreamInfo_t stStreamInfo;
MI_U32 u32OutputBufSize = 0;

//get one jpg stream from file or ipc, assign its length to u32LengthCurStream
pu8Buf = malloc(u32LengthCurStream);
//copy one jpg stream from file or ipc to pu8Buf

memset(&stInputBuf, 0, sizeof(MI_JPD_DirectInputBuf_t));
memset(&stOutputBuf, 0, sizeof(MI_JPD_DirectOutputBuf_t));
memset(&stStreamInfo, 0, sizeof(MI_JPD_StreamInfo_t));

stInputBuf.pu8InputJPGRawFrameDataVirtAddr = pu8Buf;
stInputBuf.phyAddr = 0;
stInputBuf.u32Length = u32LengthCurStream;
s32Ret = MI_JPD_QueryStreamInfo(JpdDev, JpdChn, &stInputBuf, &stStreamInfo);
if (MI_SUCCESS != s32Ret)
{
    printf("MI_JPD_QueryStreamInfo failed, s32Ret: 0x%x.\n", s32Ret);
    goto RETURN_FREE;
}
printf("stream info: w:%d h:%d format:%d bufW:%d bufH:%d\n",
        stStreamInfo.u32CurPicWidth, stStreamInfo.u32CurPicHeight,
        stStreamInfo.ePixelFormat, stStreamInfo.u32BufWidth,
        stStreamInfo.u32BufHeight);

// only support output pixel format YUYV422
stOutputBuf.ePixelFormat = E_MI_SYS_PIXEL_FRAME_YUV422_YUYV;
stOutputBuf.u32Width = stStreamInfo.u32BufWidth;
stOutputBuf.u32Height = stStreamInfo.u32BufHeight;
stOutputBuf.u32Stride[0] = stOutputBuf.u32Width * 2;
u32OutputBufSize = stOutputBuf.u32Stride[0] * stOutputBuf.u32Height;
s32Ret = MI_SYS_MMA_Alloc(NULL, u32OutputBufSize, &stOutputBuf.phyAddr[0]);
if (MI_SUCCESS != s32Ret)
{
    printf("MI_SYS_MMA_Alloc failed, s32Ret: 0x%x.\n", s32Ret);
    goto RETURN_FREE;
}

s32Ret = MI_JPD_DirectBufDecode(JpdDev, JpdChn, &stInputBuf, &stOutputBuf);
if (MI_SUCCESS != s32Ret)
{

```

```
        printf("MI_JPD_DirectBufDecode failed, s32Ret: 0x%x.\n", s32Ret);  
        goto RETURN_FREE;  
    }  
  
RETURN_FREE:  
    free(pu8Buf);  
    return s32Ret;  
}
```

➤ Related API

[MI_JPD_QueryStreamInfo](#)

3. DATA TYPE

The JPD related data types and data structures are defined as follows:

Data Structure	Description
MI_JPD_DEV	Define JPD device ID type
MI_JPD_CHN	Define JPD channel ID type
MI_JPD_ErrCode_e	Define error code enumeration.
MI_JPD_InitParam_t	Define device initialization parameter structure.
MI_JPD_ChnCreatConf_t	Define channel create configuration structure.
MI_JPD_ChnAttr_t	Define channel attribute structure.
MI_JPD_ChnStatus_t	Define channel status structure.
MI_JPD_StreamBuf_t	Define stream buffer structure.
MI_JPD_StreamInfo_t	Define stream information structure.
MI_JPD_DirectInputBuf_t	Define input buffer structure.
MI_JPD_DirectOutputBuf_t	Define output buffer structure.

3.1. MI_JPD_DEV

- Description
Define JPD device ID type .
- Syntax
#define MI_JPD_DEV MI_U32
- ※ Note
N/A.
- Related Data Type and Interface
N/A.

3.2. MI_JPD_CHN

- Description
Define JPD channel ID type.
- Syntax
#define MI_JPD_CHN MI_U32

※ Note

N/A.

➤ Related Data Type and Interface

N/A.

3.3. MI_JPD_ErrCode_e

➤ Description

Define error code enumeration.

➤ Syntax

```
typedef enum
{
    E_MI_JPD_ERR_CODE_UNKNOW = 0x0,
    E_MI_JPD_ERR_CODE_ILLEGAL_ACCESS,
    E_MI_JPD_ERR_CODE_DEC_TIMEOUT,
    E_MI_JPD_ERR_CODE_OUT_OF_MEMORY,
    E_MI_JPD_ERR_CODE_MAX
} MI_JPD_ErrCode_e;
```

➤ Member

Member	Description
E_MI_JPD_ERR_CODE_UNKNOW	Unknown error.
E_MI_JPD_ERR_CODE_ILLEGAL_ACCESS	Illegal access, e.g. uninitialized or faulty hardware.
E_MI_JPD_ERR_CODE_DEC_TIMEOUT	Decode timeout.
E_MI_JPD_ERR_CODE_OUT_OF_MEMORY	Out of memory.

※ Note

N/A.

➤ Related Data Type and Interface

N/A.

3.4. MI_JPD_InitParam_t

➤ Description

Define device initialization parameter structure.

➤ Syntax

```
typedef struct MI_JPD_InitParam_s
{
    MI_U32 u32Reserved;
} MI_JPD_InitParam_t;
```

➤ Member

Member	Description
u32Reserved	Not used, reserved for future expansion.

※ Note

N/A.

➤ Related Data Type and Interface

[MI_JPD_CreateDev](#)

3.5. MI_JPD_ChnCreatConf_t

➤ Description

Define channel create configuration structure.

➤ Syntax

```
typedef struct MI_JPD_ChnCreatConf_s
{
    MI_U32 u32StreamBufSize;
    MI_SYS_PixelFormat_e ePixelFormat;
    MI_U32 u32MaxPicWidth;
    MI_U32 u32MaxPicHeight;
} MI_JPD_ChnCreatConf_t;
```

➤ Member

Member	Description
u32StreamBufSize	The total size of stream buffer.
ePixelFormat	Image input pixel storage format, currently supports semi planar YUV420 (ie NV12)/YUYV422. Please refer to the description of MI_SYS_PixelFormat_e.
u32MaxPicWidth	The maximum width of decoded picture.
u32MaxPicHeight	The maximum height of decoded picture.

※ Note

N/A.

➤ Related Data Type and Interface

[MI_JPD_CreateChn](#)

3.6. MI_JPD_ChnAttr_t

➤ Description

Define channel attribute structure.

➤ Syntax

```
typedef struct MI_JPD_ChnAttr_s
```

```
{
    MI_U32 u32StreamBufSize;
    MI_JPD_StreamInfo_t stStreamInfo;
} MI_JPD_ChnAttr_t;
```

➤ Member

Member	Description
u32StreamBufSize	The total size of stream buffer.
stStreamInfo	Stream information, please refer to MI_JPD_StreamInfo_t description.

※ Note

N/A.

➤ Related Data Type and Interface

[MI_JPD_GetChnAttr](#)

3.7. MI_JPD_ChnStatus_t

➤ Description

Define channel status structure.

➤ Syntax

```
typedef struct MI_JPD_ChnStatus_s
{
    MI_U32 u32LeftStreamBytes;
    MI_U32 u32LeftStreamFrames;
    MI_U32 u32LeftPics;
    MI_U32 u32RecvStreamFrames;
    MI_U32 u32DecodeStreamFrames;
    MI_BOOL bChnStart;
    MI_JPD_ErrCode_e eErrCode;
} MI_JPD_ChnStatus_t;
```

➤ Member

Member	Description
u32LeftStreamBytes	The number of bytes to be decoded in the stream buffer.
u32LeftStreamFrames	The number of frames to be decoded in the stream buffer.
u32LeftPics	The number of decode pictures in the picture buffer.
u32RecvStreamFrames	The total number of received frames.
u32DecodeStreamFrames	The total number of decoded frames.
bChnStart	Whether the decoding channel has started receiving bitstream.
eErrCode	Error code.

※ Note

N/A.

- Related Data Type and Interface
[MI_JPD_GetChnStatus](#)

3.8. MI_JPD_StreamBuf_t

- Description
Define stream buffer structure.

- Syntax


```
typedef struct MI_JPD_StreamBuf_s
{
    MI_U8 *pu8HeadVirtAddr;//ring buffer first part user mode VA
    MI_PHY u64HeadPhyAddr;//ring buffer first part PA
    MI_U32 u32HeadLength;//ring buffer first part Length
    MI_U8 *pu8TailVirtAddr;//ring buffer 2nd part user mode VA
    MI_PHY u64TailPhyAddr;//ring buffer 2nd part PA
    MI_U32 u32TailLength;//ring buffer 2nd part length
    MI_U32 u32ContentLength;//current used content length
}MI_JPD_StreamBuf_t;
```

- Member

Member	Description
pu8HeadVirtAddr	The first part virtual address of bitstream buffer.
u64HeadPhyAddr	The first part physical address of bitstream buffer.
u32HeadLength	The first part length of bitstream buffer.
pu8TailVirtAddr	The second part virtual address of bitstream buffer.
u64TailPhyAddr	The second part physical address of bitstream buffer.
u32TailLength	The second part length of bitstream buffer.
u32ContentLength	Actual effective data length of bitstream buffer.

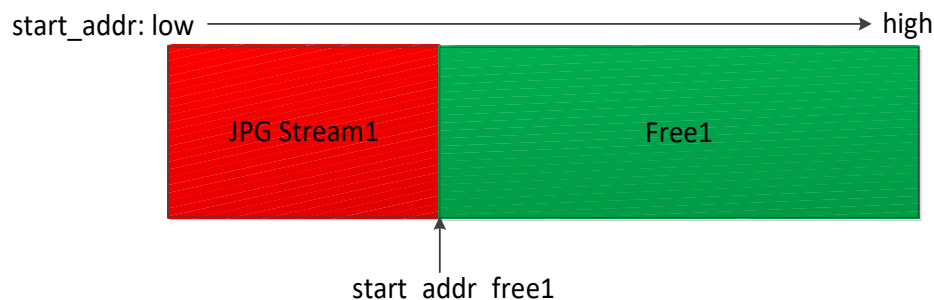
- ※ Note

- If u32TailLength is 0, that means the decoding channel returns only one bitstream buffer pu8HeadVirtAddr. Since its length is not less than the expected length from APP, APP needs to copy one jpeg coded stream into pu8HeadVirtAddr. At this time, the decoding channel stream buffer diagram may have the following situations:



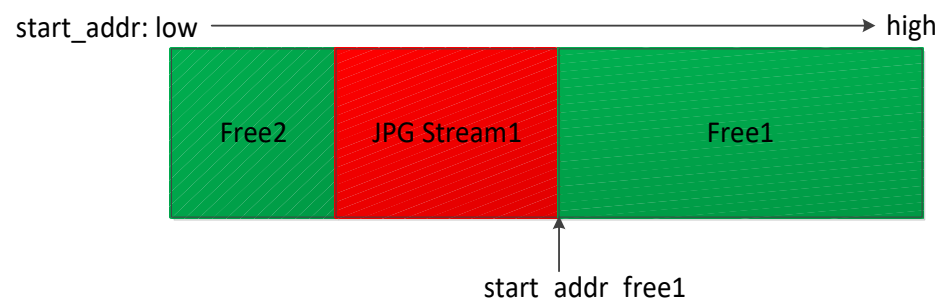
Stream buffer is free. If the length of Free1 meets the expected length u32RequiredLength, the decoding channel will assign the start_addr of stream buffer to pu8HeadVirtAddr, and assign u32RequiredLength to u32HeadLength.

- b)



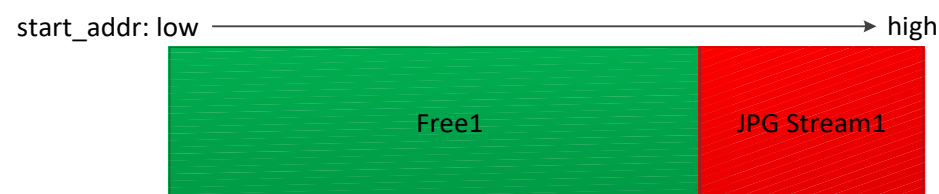
The first part of the stream buffer stores a jpeg coded stream that was previously sent in but has not yet been decoded. If the length of Free1 meets the expected length `u32RequiredLength`, the decoding channel will assign the `start_addr_free1` of stream buffer to `pu8HeadVirtAddr`, and assign `u32RequiredLength` to `u32HeadLength`.

c)



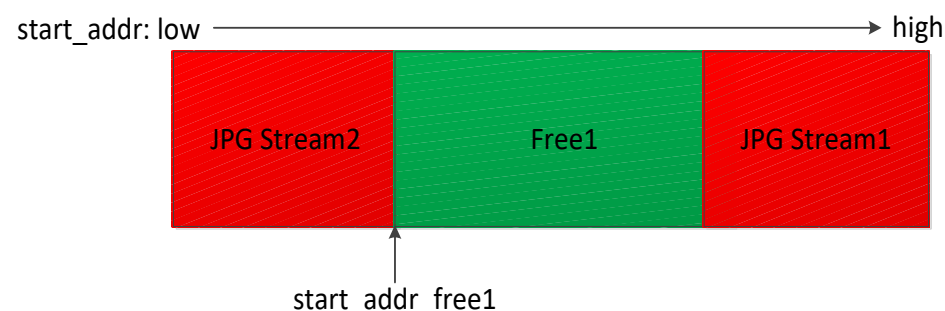
The middle part of the stream buffer stores a jpeg coded stream that was previously sent in but has not yet been decoded, with both sides free. If the length of Free1 meets the expected length `u32RequiredLength`, the decoding channel will assign the `start_addr_free1` of stream buffer to `pu8HeadVirtAddr`, and assign `u32RequiredLength` to `u32HeadLength`.

d)



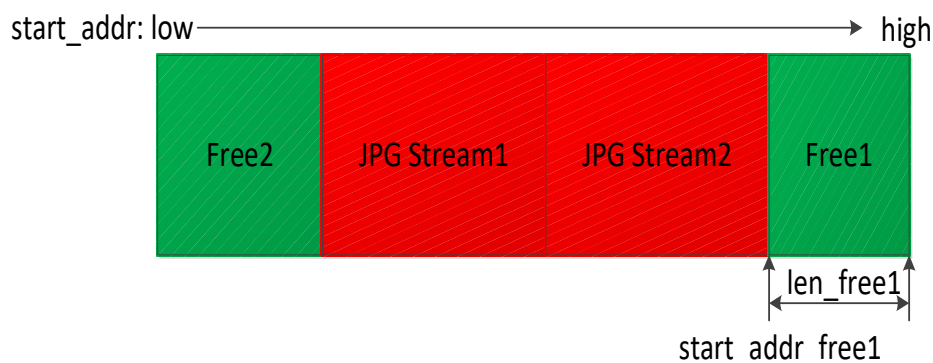
The last part of the stream buffer stores a jpeg coded stream that was previously sent in but has not yet been decoded. If the length of Free1 meets the expected length `u32RequiredLength`, the decoding channel will assign the `start_addr` of stream buffer to `pu8HeadVirtAddr`, and assign `u32RequiredLength` to `u32HeadLength`.

e)



Only the middle part of the stream buffer is free. If the length of Free1 meets the expected length `u32RequiredLength`, the decoding channel will assign the `start_addr_free1` of stream buffer to `pu8HeadVirtAddr`, and assign `u32RequiredLength` to `u32HeadLength`.

- If `u32TailLength` is not 0, that means the decoding channel returns two bitstream buffers. Since the total length of the sum of the two bitstream buffers meets the expected length, APP needs to copy a frame of coded stream to `pu8HeadVirtAddr` and `pu8TailVirtAddr` in order. At this time, the decoding channel stream buffer diagram may have the following situations:



The middle part of the stream buffer stores a jpeg coded stream that was previously sent in but has not yet been decoded, with both sides free. Since only the sum of the total length of the first and last two free buffers can meet the expected length, the decoding channel will assign the `start_addr_free1` to `pu8HeadVirtAddr`, assign `len_free1` to `u32HeadLength`, assign `start_addr` to `pu8TailVirtAddr`, and assign `(u32RequiredLength - u32HeadLength)` to `u32TailLength`.

- `u32ContentLength` means actual effective data length of bitstream. If the actual coded stream length transmitted by app is less than the expected length `u32RequiredLength` of [MI_JPD_GetStreamBuf](#), APP needs to assign the actual length to `u32ContentLength`, then call [MI_JPD_PutStreamBuf](#).

➤ Related Data Type and Interface

[MI_JPD_GetStreamBuf](#)

[MI_JPD_PutStreamBuf](#)

[MI_JPD_DropStreamBuf](#)

3.9. MI_JPD_StreamInfo_t

➤ Description

Define stream information structure.

➤ Syntax

```
typedef struct MI_JPD_StreamInfo_s
{
    MI_U32 u32CurPicWidth;
    MI_U32 u32CurPicHeight;
    MI_SYS_PixelFormat_e ePixelFormat;
```

```
MI_U32 u32BufWidth;
MI_U32 u32BufHeight;
} MI_JPD_StreamInfo_t;
```

➤ Member

Member	Description
u32CurPicWidth	The current effective width of decoded picture.
u32CurPicHeight	The current effective height of decoded picture.
ePixelFormat	Input pixel format, with support for semi planar YUV420(NV12)/YUYV422. Please refer to MI_SYS_PixelFormat_e description.
u32BufWidth	The actual width of the output buffer.
u32BufHeight	The actual height of the output buffer.

※ Note

- If the output buffer size is not clear, APP needs to call this interface before calling MI_JPD_DirectBufDecode. The output buffer size is calculated by u32BufWidth/u32BufHeight, while u32CurPicWidth/u32CurPicHeight refers to the actual effective width/height of the image.

➤ Related Data Type and Interface

[MI_JPD_ChnAttr_t/MI_JPD_QueryStreamInfo](#)

3.10. MI_JPD_DirectInputBuf_t

➤ Description

Define input buffer structure.

➤ Syntax

```
typedef struct MI_JPD_DirectInputBuf_s
{
    MI_U8 *pu8InputJPGRawFrameDataVirtAddr;
    MI_PHY phyAddr;
    MI_U32 u32Length;
}MI_JPD_DirectInputBuf_t;
```

➤ Member

Member	Description
pu8InputJPGRawFrameDataVirtAddr	The virtual address that stores JPG bitstream data.
phyAddr	Optional, the physical address that stores JPG bitstream data, default value is 0.
u32Length	The length of JPG bitstream data.

※ Note

- The input buffer cannot be gotten from MI_JPD_GetStreamBuf, but through the system application.
- If the coded stream is always sent and decoded in this way, the parameter

u32StreamBufSize of [MI_JPD_CreateChn](#) can be set to 0, to spare the decoding channel the effort of allocating the stream buffer.

- The purpose of synchronous decoding is achieved through the [MI_JPD_DirectBufDecode](#) interface. Pass [MI_JPD_GetStreamBuf](#)/ [MI_JPD_PutStreamBuf](#) for asynchronous decoding process.

➤ Related Data Type and Interface

[MI_JPD_QueryStreamInfo](#)/[MI_JPD_DirectBufDecode](#)

3.11. [MI_JPD_DirectOutputBuf_t](#)

➤ Description

Define output buffer structure.

➤ Syntax

```
typedef struct MI_JPD_DirectOutputBuf_s
{
    MI_SYS_PixelFormat_e ePixelFormat;
    MI_U32 u32Width;
    MI_U32 u32Height;
    MI_U32 u32Stride[2];
    MI_PHY phyAddr[2];
}MI_JPD_DirectOutputBuf_t;
```

➤ Member

Member	Description
ePixelFormat	Output pixel format, only supports YUYV422.
u32Width	The actual width of output buffer.
u32Height	The actual height of output buffer.
u32Stride	u32Stride[0]: the stride of Y data; u32Stride[1]: the stride of UV data.
phyAddr	phyAddr[0]: the start physical address of Y data; phyAddr[1]: the start physical address of UV data.

※ Note

- The output pixel format only supports YUYV422.

➤ Related Data Type and Interface

[MI_JPD_DirectBufDecode](#)

4. ERROR CODE

The error code is shown in the following table:

Table 4-2: JPD API Error Code

Error Code	Macro Definition	Description
0xA0202001	MI_ERR_JPD_INVALID_DEVID	Invalid Device ID.
0xA0202002	MI_ERR_JPD_INVALID_CHNID	Invalid Channel ID.
0xA0202003	MI_ERR_JPD_ILLEGAL_PARAM	Illegal parameter or inputted parameter exceeding channel decoding capability.
0xA0202004	MI_ERR_JPD_CHN_EXIST	Channel to be created already exists.
0xA0202005	MI_ERR_JPD_CHN_UNEXIST	Channel does not exist.
0xA0202006	MI_ERR_JPD_NULL_PTR	Null pointer.
0xA0202007	MI_ERR_JPD_NOT_CONFIG	Not configured before use.
0xA0202008	MI_ERR_JPD_NOT_SUPPORT	This operation or function is not supported.
0xA0202009	MI_ERR_JPD_NOT_PERM	Illegal operation
0xA020200C	MI_ERR_JPD_NOMEM	Memory allocation fails, such as insufficient memory
0xA020200D	MI_ERR_JPD_NOBUF	Buffer allocation fails, like the requested data buffer is too large
0xA020200E	MI_ERR_JPD_BUF_EMPTY	No data in buffer.
0xA020200F	MI_ERR_JPD_BUF_FULL	Buffer full.
0xA0202010	MI_ERR_JPD_SYS_NOTREADY	The system is not initialized or the dependent modules are not loaded.
0xA0202011	MI_ERR_JPD_BADADDR	Address error.
0xA0202012	MI_ERR_JPD_BUSY	System busy.
0xA0202013	MI_ERR_JPD_CHN_NOT_START	Channel not started or already stopped.
0xA0202014	MI_ERR_JPD_CHN_NOT_STOP	Channel cannot be closed before code stream reception is stopped.
0xA0202015	MI_ERR_JPD_NOT_INIT	Device is not initialized.
0xA0202016	MI_ERR_JPD_INITED	Device to be initialized is already initialized.
0xA020201C	MI_ERR_JPD_CHN_NO_CONTENT	Channel has no data.
0xA020201F	MI_ERR_JPD_FAILED	Failed.

5. PROCFS INTRODUCTION

5.1. cat

➤ Debug info

#cat /proc/mi_modules/mi_jpd/mi_jpdN

N represents different devices, and the values of N for different chips are as follows:

Table 5-1 N value of different chips

Chip	N
Tiramisu	0
Muffin	0~3

➤ Debug info analysis

The printing is divided into two parts, separated by Private JpdN Info. The upper half is common information, and the lower half is jpd module information.

It mainly records the usage and configuration attributes of the decoding channel, which can be used to check the attribute configuration and the working status of the current channel to facilitate debugging.

➤ Parameter Description

Parameter		Description
DEV Info	DevID	Hardware device id, refer to Table 5-1
	WorkingTaskCnt	The number of tasks being processed by the device
CHN Info	ChnID	Channel ID
	Start	Enable state: 0: Disable 1: Enable
	DecState	Decoding state: 0: Parse header 1: Decoding 2: End of decoding
	Width	Current decoding image width
	Height	Current decoding image height
	BufW	The output buffer width that needs to be allocated for the current decoding image
	BufH	The output buffer height that needs to be allocated for the current decoding image
	StreamBufSize	Code stream input buffer pool size, unit: byte

Parameter		Description
	MinAllocSize	Allow the input buffer min size to be allocated successfully
	WorkingTaskCnt	The number of tasks being processed by the channel
	IsrDoneCnt	Isr count of done type
	IsrOtherCnt	Isr count of other type
	Poll	Time to run OnPollingAsyncOutputTaskConfig, unit: ms
	Enque	Time to run EnqueueAsyncOutputTask, unit: ms
	Isr	Time to run isr, unit: ms
	Check	Time to run CheckOutputTaskStatus, unit: ms
	Dequeue	Time to run DequeueOutputTask, unit: ms
	P->E	Time from Poll to Enque, unit: ms
	E->I	Time from Enque to Isr, unit: ms
	I->C	Time from Isr to Check, unit: ms
	C->D	Time from Check to Dequeue, unit: ms
	P->D	Time from Poll to Dequeue, unit: ms
	SmoothDecodeTime	Average decode time, unit: ms
CHN INPUT Info	ChnID	Channel ID
	SendCnt	The count of stream sent by App
	LeftCnt	The count of stream left by Input buffer pool
	SendStreamSize	Total size of stream sent by App
	LeftStreamBytes	Total size of stream left by Input buffer pool
	EnqueueCnt	The total number of successful executions of EnqueueAsyncOutputTask
	DirectCnt	The number of times that MI_JPD_DirectBufDecode is called to trigger decoding
	DropCnt	The total number of failed executions of EnqueueAsyncOutputTask
	RewindCnt	The total number of executions of EnqueueAsyncOutputTask rewind
	ifps	Frame rate of App input stream
	MinIntvl	The minimum interval of APP input stream, unit: ms
	MaxIntvl	The maximum interval of APP input stream, unit: ms
	SumIntvl	The accumulation interval of APP input stream, unit: ms
	AvgIntvl	The average interval of APP input stream, unit: ms
	Diff	The current interval of the APP input stream, unit: ms

Parameter		Description
CHN OUTPUT Info	ChnID	Channel ID
	DecCnt	The total number of successful decoding
	DirectCnt	The total number of successful decoding calls MI_JPD_DirectBufDecode
	DropCnt	The total number of failed decoding
	ofps	Output frame rate
	MinIntvl	The minimum interval of APP output stream, unit: ms
	MaxIntvl	The maximum interval of APP output stream, unit: ms
	SumIntvl	The accumulation interval of APP output stream, unit: ms
	AvgIntvl	The average interval of APP output stream, unit: ms
	Diff	The current interval of the APP output stream, unit: ms

5.2. echo

Function	Dump the bitstream buffer of the specified channel to the specified path
Command	echo dumpsb [chn] [bEnable] [path] > /proc/mi_modules/mi_jpd/mi_jpd[dev]
Parameter Description	[chn]: Channel ID [bEnable]: 1: enable dump; 0: disable dump [path]: Path to save the file [dev]: Device ID, refer to Table 5-1
Example	echo dumpsb 3 1 /mnt > /proc/mi_modules/mi_jpd/mi_jpd0 Dump bitstream buffer of channel 3 to /mnt

Function	Dump the decoded output data frame of the specified channel to the specified path
Command	echo dumpfb [chn] [bEnable] [path] > /proc/mi_modules/mi_jpd/mi_jpd[dev]
Parameter Description	[chn]: Channel ID [bEnable]: 1: enable dump; 0: disable dump [path]: Path to save the file [dev]: Device ID, refer to Table 5-1
Example	echo dumpfb 3 1 /mnt > /proc/mi_modules/mi_jpd/mi_jpd0 Dump output data frame of channel 3 to /mnt